

HalluScope: Fine-grained Hallucination Diagnosis for Multimodal Large Language Models

Weilin Jin
Peking University
Beijing, China

Mingyu Wang
Peking University
Beijing, China

Wenbo Li*
Joy Future Academy
Beijing, China

Haoyang Huang
Joy Future Academy
Beijing, China

Yifan Wu†
Peking University
Beijing, China

Ying Li†
Peking University
Beijing, China

Gang Huang
Peking University
Beijing, China

Zhonghai Wu
Peking University
Beijing, China

Abstract

Although Multimodal Large Language Models have achieved strong performance across a wide range of vision-language tasks, they still suffer from hallucinations, where model outputs become inconsistent with the visual content, textual context, or commonsense knowledge. Existing studies primarily address this problem through coarse-grained detection. However, these approaches often provide insufficient diagnostic information for understanding hallucination types and supporting downstream hallucination mitigation. To bridge this gap, we propose fine-grained hallucination diagnosis for MLLMs, a new unified task that jointly performs hallucination detection, classification, and interpretable explanation generation. We develop an automated data generation pipeline and construct HalluScope-30K, a large-scale diagnostic dataset covering eight sources and five task categories. Based on this dataset, we design a multi-granular joint reward function and train two diagnosis models, HalluScope-4B and HalluScope-8B, which achieve state-of-the-art performance on both the MHALO benchmark and our fine-grained hallucination classification benchmark. Notably, detection and classification are mutually beneficial under joint optimization. Furthermore, diagnosis-driven feedback experiments show that the fine-grained diagnostic explanations produced by our model effectively guide target models to correct their hallucinations, with full diagnosis substantially outperforming all baselines on both Qwen3-VL-8B-Instruct and LLaVA-1.5-7B.

CCS Concepts

• Computing methodologies → Computer vision problems.

Keywords

Multimodal Large Language Models, Hallucination Diagnosis, Reinforcement Learning, Hallucination Mitigation

1 INTRODUCTION

Multimodal Large Language Models (MLLMs) [2, 14, 25, 55] have achieved strong performance across a wide range of vision-language

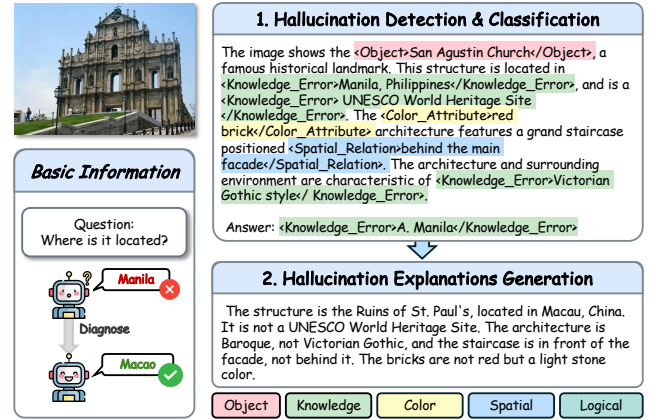


Figure 1: Example of fine-grained hallucination diagnosis. Given an image-question pair and an MLLM response, HalluScope (1) detects hallucinated spans with type classification, and (2) generates explanations with correction suggestions.

tasks, including image understanding [6], visual question answering [1, 21], and visual reasoning [33, 43]. Despite these advances, hallucination remains a persistent challenge, where model outputs become inconsistent with the visual content, textual context, or commonsense knowledge [3, 26, 34]. Such errors undermine the reliability of MLLM outputs, particularly in high-stakes scenarios such as healthcare, autonomous driving, and scientific analysis [9, 24]. Consequently, identifying and mitigating hallucinations has become a critical challenge for MLLMs.

Existing studies primarily address hallucination through coarse-grained detection or feedback, typically formulating it as a binary classification problem [7, 49, 53]. However, such methods only determine whether a response contains hallucinations, without localizing hallucinated spans, identifying their types, or explaining the underlying causes, thereby providing insufficient diagnostic information for effective hallucination analysis and mitigation.

To address these limitations, recent work has explored Fine-grained Hallucination Detection (FHD) [4], which localizes hallucinated spans within the response at the token level. Compared

*Project leader.

†Corresponding author.

with coarse-grained methods, FHD provides more informative signals for hallucination analysis, enabling high-quality data annotation [16, 49], model alignment [12], and feedback generation.

Despite these advances, existing FHD methods still focus primarily on span-level detection, without further analyzing the types or causes of identified hallucinations. Yet different hallucination types arise from fundamentally different failure mechanisms [31] and require distinct mitigation strategies. For instance, **Perceptual Hallucinations** such as object recognition errors can often be mitigated by re-examining the images, whereas **Reasoning Hallucinations** involving spatial reasoning or complex semantic relations require more sophisticated verification mechanisms. Fine-grained hallucination classification can reveal such error patterns, enabling targeted analysis and more effective mitigation. Beyond classification, interpretable explanations that describe why a hallucination occurs and how it may be corrected provide actionable guidance for downstream correction. Therefore, hallucination detection, classification, and explanation generation are not independent but mutually beneficial, making their joint optimization essential for both comprehensive diagnosis and effective mitigation.

Motivated by these observations, we propose **fine-grained hallucination diagnosis** for MLLMs, a task that unifies hallucination detection, classification, and interpretable explanation generation. To support this task, we develop a multimodal hallucination diagnosis data generation pipeline and construct **HalluScope-30K**, a large-scale dataset for fine-grained hallucination diagnosis. Based on this dataset, we design a multi-granular joint reward function to jointly optimize hallucination detection and classification, and train two diagnosis models, **HalluScope-4B** and **HalluScope-8B**.

Since hallucination diagnosis is inherently a multi-stage task, we evaluate each stage independently. For fine-grained hallucination detection, we evaluate our models on the MHALO benchmark. Compared with representative MLLMs and specialized hallucination detection baselines, our models achieve state-of-the-art performance on multiple subsets, reaching an average $F1_M$ of **64.03** and $F1_{IoU}$ of **57.57**. For hallucination classification, we further construct a fine-grained benchmark with 733 samples covering five task categories including general perception, mathematical reasoning, OCR, spatial reasoning, and hallucination detection. On this benchmark, our method also achieves state-of-the-art performance, with $F1_{Macro}$ of **51.66** and $F1_{Micro}$ reaching **59.60**. For hallucination mitigation, we conduct diagnosis-driven feedback experiments on two target models. The results reveal that increasing diagnostic granularity progressively improves mitigation effectiveness: on Qwen3-VL-8B-Instruct, full diagnosis raises accuracy from **68.76%** to **81.17%**, and on LLaVA-1.5-7B, from **33.02%** to **50.34%**, substantially outperforming detection-only and self-correction baselines. These results collectively confirm that detection, classification, and explanation are mutually beneficial: joint optimization improves each individual stage, and richer diagnosis leads to more effective mitigation. Our contributions are as follows:

- We propose a new task, **fine-grained hallucination diagnosis** for MLLMs, which unifies hallucination detection, classification, and interpretable explanation generation, and find that the three stages are mutually beneficial under joint optimization.
- We develop an automated data generation pipeline and construct **HalluScope-30K**, a large-scale dataset that provides fine-grained supervision for hallucination diagnosis.
- We design a multi-granular joint reward function that enables detection and classification to benefit each other during training, and train two diagnosis models, **HalluScope-4B** and **HalluScope-8B**.
- We construct a fine-grained hallucination classification benchmark and achieve state-of-the-art performance on both the MHALO benchmark and our benchmark. We further demonstrate that increasing diagnostic granularity progressively improves hallucination mitigation across two target models.

2 RELATED WORK

2.1 Hallucination in MLLMs

Hallucination in multimodal large language models (MLLMs), where the generated content is inconsistent with the visual input, textual context, or established world knowledge [8, 26], has attracted increasing attention. In MLLMs, such inconsistencies are typically attributed to strong language priors that encourage fluent but visually ungrounded continuations, and statistical biases from pre-training data that lead models to rely on spurious correlations [23].

Prior studies have explored hallucination detection by identifying inconsistencies between model outputs and visual or textual evidence, progressing from coarse-grained response-level judgments to finer-grained analysis such as factual consistency assessment, claim verification, and span-level detection [7, 35, 41, 50, 52]. In parallel, another line of research focuses on hallucination mitigation through training-time alignment, external verification, inference-time decoding control, and internal model editing [23, 27, 38, 42, 47–49, 54, 56]. While effective mitigation often relies on accurate detection signals, most existing methods treat detection and mitigation as separate stages without providing unified diagnostic feedback that integrates error localization, type classification, and causal explanation. Our work addresses this gap by proposing fine-grained hallucination diagnosis as a unified task.

2.2 Fine-grained Hallucination Detection and Mitigation for MLLMs

Recent studies on hallucination in MLLMs have increasingly moved from coarse-grained response-level judgment toward fine-grained analysis, which aims to provide more precise supervision for hallucination detection and mitigation. Early efforts mainly decompose model outputs into smaller textual or semantic units, such as clauses, sentences, or claims. For example, M-HalDetect [16] performs clause-level hallucination detection, Xiao et al. [46] operate at the sentence level, and Pelican [35] verifies hallucinations at the claim level through claim decomposition and grounding.

More recent work pushes fine-grained hallucination detection and mitigation toward token-level modeling [4, 10–12, 44, 57]. Whitehead et al. [44] formulate hallucination detection as a sequence labeling problem to directly localize hallucinated content at fine granularity. MHALO [4] further trains a dedicated model for token-level hallucination detection with coarse-grained type classification, but does not provide fine-grained classification or diagnostic explanations for downstream mitigation. In addition,

MetaToken [11], TLDR [12], and ECD [10] estimate token-level hallucination likelihood during generation through meta-classification, reward modeling, and decoding-time probability estimation, respectively. Meanwhile, MTRE [57] suggests that hallucination signals may emerge progressively across multiple tokens or semantic levels. Overall, fine-grained hallucination signals are increasingly integrated with downstream mitigation. However, existing methods primarily address span-level localization, while hallucination classification remains coarse-grained and diagnostic explanations are largely absent, limiting the actionable value of detection outputs.

3 PRELIMINARY

3.1 Hallucination Diagnosis Task Definition

We formulate fine-grained hallucination diagnosis as a unified task that encompasses hallucination detection, hallucination classification, and interpretable explanation generation. Given a multimodal input consisting of an image, a textual query, and the model’s response, the task aims to localize hallucinated spans in the response, classify each span into a predefined hallucination type, and generate interpretable explanations that describe the causes of hallucination and suggest possible correction strategies.

Specifically, for hallucination detection, the model identifies and marks hallucinated spans within the response through structured XML annotations. For hallucination classification, we adopt the taxonomy proposed in MHALO [4], which organizes hallucinations into two high-level categories: *perception* and *reasoning*. Perception-level hallucinations arise from errors in visual understanding or information extraction from images, whereas reasoning-level hallucinations originate from incorrect inference or logical reasoning based on perceived information (the full taxonomy with 12 fine-grained types is provided in Appendix). For explanation generation, the model produces natural language descriptions that explain why each hallucination occurs and suggest how it may be corrected.

3.2 Hallucination Diagnosis Formulation

Notation for data construction: In the data generation pipeline, each sample is represented as $x_k = (i_k, d_k, q_k, g_k, o_k, y_k^*)$, where i_k denotes the visual input, d_k the dataset source, q_k the question, and g_k the ground-truth answer. Given (i_k, q_k) , the base model first generates a response $o_k \sim p_\theta(\cdot | i_k, q_k)$. Based on the consistency between o_k and g_k , the response is further processed through hallucination injection or hallucination annotation to produce the final hallucination-diagnosed response y_k^* . Each diagnosed span in y_k^* is assigned one of C predefined hallucination types.

Notation for evaluation: To evaluate hallucination diagnosis, we denote the set of predicted spans as $P = \{p_i\}_{i=1}^n$ and the set of ground-truth spans as $G = \{g_j\}_{j=1}^m$, where n and m denote the numbers of predicted and ground-truth spans, respectively. Each span corresponds to a contiguous token segment in the response and is represented by its start and end boundaries:

$$p_i = [s_i^p, e_i^p], \quad g_j = [s_j^g, e_j^g],$$

where s_i^p and e_i^p denote the start and end positions of the i -th predicted span, and s_j^g and e_j^g denote the start and end positions of the j -th ground-truth span. Accordingly, the span lengths are

defined as

$$|p_i| = e_i^p - s_i^p + 1, \quad |g_j| = e_j^g - s_j^g + 1.$$

We further use $T(p_i)$ and $T(g_j)$ to denote the token sets contained in the predicted span p_i and the ground-truth span g_j , respectively. Their token overlap is defined as:

$$|T(p_i) \cap T(g_j)|,$$

which counts the number of shared tokens between the two spans.

4 METHOD

Figure 2 presents the HalluScope framework. Starting from eight datasets across five task categories, the framework generates model responses and constructs fine-grained hallucination labels through two complementary strategies: *hallucination injection* for correctly answered samples and *hallucination annotation* for incorrectly answered ones. After multi-stage quality control, the framework yields HalluScope-30K with nearly 30,000 diagnostic examples. Based on this dataset, we design a multi-granular joint reward function and train two diagnosis models: HalluScope-4B and HalluScope-8B.

4.1 Pipeline of Data Generation

4.1.1 Data Collection. The HalluScope framework begins with the collection of eight datasets spanning five task categories: (1) *general perception*: MMBench [28] and MMStar [5]; (2) *mathematical reasoning*: Geo170K [13] and MathV360K [36]; (3) *ocr*: OCRBench [29]; (4) *spatial reasoning*: RealWorldQA [45]; and (5) *hallucination detection*: M-HalDetect [16] and RLHF-V [49]. Since these datasets differ in format and annotation, we first perform standardized preprocessing: unifying data fields, removing duplicates, and filtering invalid samples (e.g., missing answers). Each processed sample follows the notation x_k defined in Section 3.2.

Given the unified dataset, we use the base model (before post-training) to generate responses for each sample. Since the original ground-truth answers are typically brief and lack sufficient detail for fine-grained annotation, we regenerate longer, more informative responses to provide richer context for hallucination diagnosis. Unlike approaches that rely on strong closed-source models to expand ground-truth answers, constructing hallucination data from the base model’s actual outputs better preserves the realism of synthesized samples, as they reflect the model’s genuine hallucination behaviors rather than artificially simulated ones. This also improves data diversity, since the model exhibits varied hallucination patterns across different tasks and samples.

4.1.2 Hallucination Label Generation. After obtaining model responses, we construct fine-grained hallucination labels by partitioning samples based on the consistency between the model response and the ground-truth answer. For **correctly answered samples**, we employ a closed-source model (Gemini-3-Pro) to perform *hallucination injection*: analyzing the original response and introducing targeted modifications tailored to specific hallucination types while maintaining contextual coherence. The injected hallucinations are constrained to be diverse across types and natural within context, ensuring broad coverage of the hallucination taxonomy. For **incorrectly answered samples**, we employ the same closed-source model to perform *hallucination annotation*: localizing hallucinated

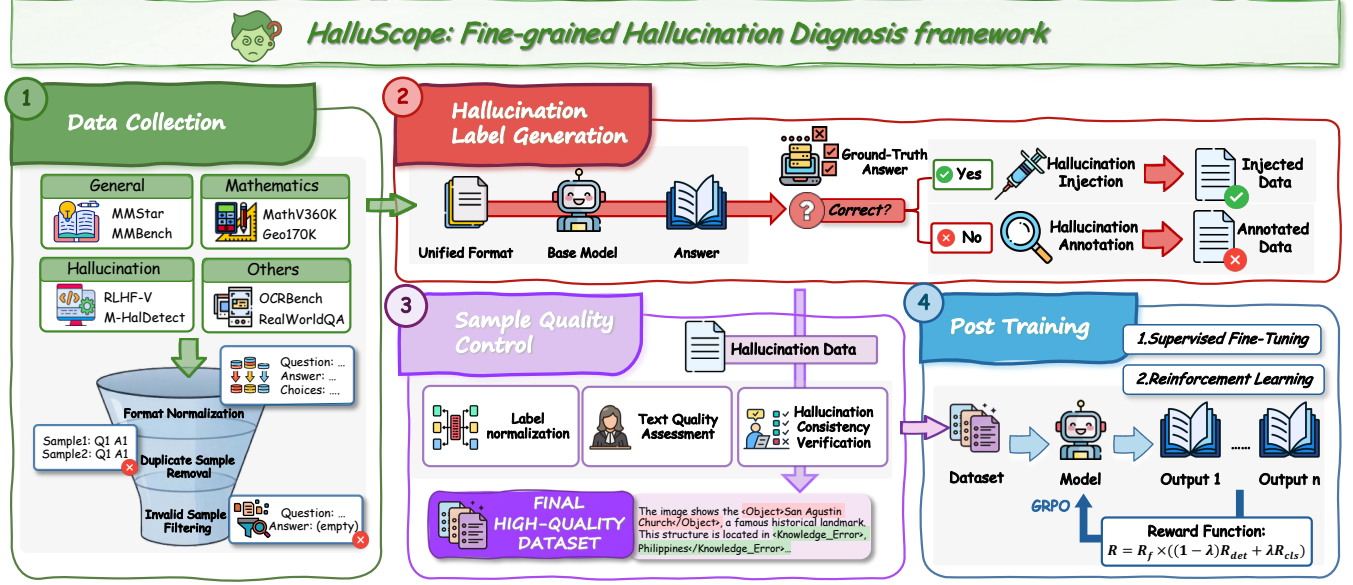


Figure 2: Overview of the HalluScope data generation pipeline and training pipeline.

spans in the response and assigning corresponding type labels. Since these responses already contain genuine hallucinations produced by the base model, annotation directly captures realistic hallucination behaviors without artificial simulation. By combining both strategies, we preserve hallucination samples from genuine model outputs while also synthesizing new ones from correct responses, jointly yielding fine-grained supervision signals that specify both hallucination locations and types.

4.1.3 Sample Quality Control. Finally, we perform systematic quality control through a three-stage pipeline: (1) *label normalization*, which maps non-standard hallucination type labels to the standardized set of 12 hallucination types through rule-based mappings (e.g., case normalization and synonym resolution); (2) *text quality assessment*, which uses an LLM-as-Judge method to score clarity, fluency, and coherence, retaining only samples above predefined thresholds; and (3) *hallucination consistency verification*, which checks format correctness, type validity, semantic plausibility, and contextual consistency of the generated hallucination labels. Only samples passing all three stages are included in the final dataset.

Through this pipeline, we construct HalluScope-30K, containing nearly 30,000 samples across five task categories and eight sources, each with fine-grained hallucination diagnosis labels. Dataset statistics and quality control criteria are provided in Appendix.

4.2 Training of HalluScope Models

Based on HalluScope-30K, we train two diagnosis models through a two-stage paradigm: supervised fine-tuning (Section 4.2.1) followed by reinforcement learning (Section 4.2.2).

4.2.1 Supervised Fine-Tuning. In the supervised fine-tuning stage, we use HalluScope-30K as supervision to train the base model with LoRA [18], enabling it to generate structured diagnostic outputs that unify hallucination detection, classification, and interpretable

feedback generation. This stage serves as a cold-start initialization for subsequent reinforcement learning: the model learns to follow the required output format and acquires preliminary hallucination detection and classification abilities, reducing ineffective exploration during RL optimization [51].

4.2.2 Reinforcement Learning. In the reinforcement learning stage, we apply Group Relative Policy Optimization (GRPO) [17] to further optimize the SFT-initialized model. For each sample x_k , the policy generates K candidate outputs $\{y_k\}_{k=1}^K$, each scored by a multi-granular reward $R(y_k)$ that jointly evaluates format compliance, span-level detection quality, and type classification accuracy. Using the notation defined in Section 3.2, the reward is formulated as:

$$R(y_k) = \mathbb{I}_{\text{format}}((1 - \lambda)r_{\text{det}}(y_k) + \lambda r_{\text{cls}}(y_k)) \quad (1)$$

Format Reward. $\mathbb{I}_{\text{format}}$ is a binary indicator function that equals 1 only when the diagnostic output satisfies all predefined formatting constraints. Specifically, the entire result must be enclosed within `<Tagged_Text>` tags, and each hallucinated span must be annotated using `<hallucination type="TYPE">` tags. All tags must comply with XML syntax rules, and each type attribute must belong to the predefined hallucination types. This gating mechanism ensures that only structurally valid outputs receive positive diagnostic rewards, with the preceding SFT stage providing a warm start for format compliance.

Hallucination Detection Reward. Hallucination detection reward evaluates the model’s ability to detect hallucinations and serves as the most critical component of the overall reward function. Since the fine-grained hallucination diagnosis task requires the model not only to identify all hallucinations, but also to localize each hallucinated span accurately, the detection reward is designed to jointly consider both coverage and precision of the predicted spans. To this end, we define a composite detection reward as:

$$r_{\text{det}}(y_k) = r_{\text{content}}(y_k) \cdot r_{\text{count}}(y_k) \quad (2)$$

Here r_{content} measures span matching quality and r_{count} penalizes deviations in the predicted number of spans. Specifically, r_{content} computes the average IoU over optimal span pairs matched by the Hungarian algorithm (following the notation in Section 5.1.1):

$$r_{\text{content}}(y_k) = \frac{1}{|\hat{M}|} \sum_{(i,j) \in \hat{M}} \text{IoU}(p_i, g_j) \quad (3)$$

where $\hat{M}$ maximizes the total IoU across all matched span pairs. The count reward r_{count} measures the consistency between the number of predicted and ground-truth hallucinated spans. Since relying solely on content matching may not sufficiently penalize models that produce too many or too few spans, we introduce a Gaussian-kernel smoothing function that yields a maximum reward of 1 when the counts match exactly and decays smoothly as the discrepancy increases:

$$r_{\text{count}}(y_k) = \exp\left(-\frac{(n-m)^2}{m}\right), \quad (4)$$

where $n = |P|$ and $m = |G|$ denote the numbers of predicted and ground-truth hallucinated spans, respectively.

Hallucination Classification Reward. In the hallucination diagnosis task, hallucination classification is built upon successful hallucination detection. Therefore, when designing the classification reward, we still take the IoU-based matching result as the foundation. After obtaining the optimal matching $\hat{M}$, we further examine whether the hallucination type labels of each matched pair are consistent, and reward the classification result accordingly. Based on this idea, the classification reward is defined as follows:

$$r_{\text{cls}}(y_k) = \frac{1}{|\hat{M}|} \sum_{(i,j) \in \hat{M}} \text{IoU}(p_i, g_j) \cdot \mathbf{1}[t_i = t_j] \quad (5)$$

This multi-granular reward design provides fine-grained and stable optimization signals for hallucination diagnosis. Specifically, the format reward enforces structurally valid outputs, the detection reward promotes accurate hallucination span localization, and the classification reward encourages distinguishing fine-grained hallucination types. Compared with reward designs based solely on final matching results, this hierarchical formulation jointly optimizes output structure, span localization, and type classification, reducing reward sparsity and improving training stability. Based on this training strategy, we obtain two hallucination diagnosis models: HalluScope-4B and HalluScope-8B.

5 EXPERIMENTS

5.1 Experimental Setup

5.1.1 Evaluation Metrics. We use four metrics to evaluate hallucination diagnosis. For fine-grained hallucination detection, we adopt and refine $\mathbf{F1}_M$ and $\mathbf{F1}_{\text{IoU}}$ from MHALO [4] by introducing positional consistency constraints. For hallucination classification, we use $\mathbf{F1}_{\text{Macro}}$ and $\mathbf{F1}_{\text{Micro}}$ scores.

Partial Match Metric $\mathbf{F1}_M$. To measure the matching quality between predicted and ground-truth spans, we adopt the recall-based partial match score (PM_R) proposed by Jafari et al. [20]. However,

the original PM_R only considers token overlap, which may produce incorrect matches when identical tokens appear at different positions. We therefore introduce a *positional consistency constraint*: a partial match is valid only when the predicted and ground-truth spans overlap in their positional boundaries. The refined PM_R is defined as:

$$\text{PM}_R(g_j) = \begin{cases} 1, & \exists p_i \in P, p_i = g_j, \\ \frac{|T(p_i) \cap T(g_j)|}{|T(g_j)|}, & \exists p_i \in P, \max(s_i^p, s_j^g) \leq \min(e_i^p, e_j^g), \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

If multiple p_i overlap with g_j , the best-scoring one is selected. The precision-based score PM_P is defined analogously. The overall recall and precision are $\text{Rec}_M = \frac{1}{m} \sum_j \text{PM}_R(g_j)$ and $\text{Prec}_M = \frac{1}{n} \sum_i \text{PM}_P(p_i)$, and $\mathbf{F1}_M$ is their harmonic mean.

Token-aware IoU Metric $\mathbf{F1}_{\text{IoU}}$. While $\mathbf{F1}_M$ captures inclusion relationships between spans, it does not quantify the content-level overlap. We refine the $\mathbf{F1}_{\text{IoU}}$ metric from MHALO by incorporating token-content consistency into the IoU computation. The token-aware IoU is computed only when positional overlap exists:

$$\text{IoU}_t(p_i, g_j) = \begin{cases} \frac{|T(p_i) \cap T(g_j)|}{|T(p_i) \cup T(g_j)|}, & \max(s_i^p, s_j^g) \leq \min(e_i^p, e_j^g), \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

This formulation reduces overestimation from boundary coincidence when predicted and ground-truth spans have little actual token overlap. A match is valid only when $\text{IoU}_t(p_i, g_j) \geq \delta$ ($\delta = 0.5$). The optimal one-to-one matching is obtained via the Hungarian algorithm [22]:

$$\hat{M} = \max_{M \in \mathcal{M}} \sum_{(i,j) \in M} \mathbf{1}(\text{IoU}_t(p_i, g_j) \geq \delta), \quad (8)$$

where $\mathcal{M}$ denotes the set of all one-to-one bipartite matchings between P and G :

$$\mathcal{M} = \{M \subseteq P \times G \mid \forall (p_i, g_j), (p_{i'}, g_{j'}) \in M, i \neq i', j \neq j'\}. \quad (9)$$

The final score is $\mathbf{F1}_{\text{IoU}} = 2|\hat{M}|/(|P| + |G|)$, which balances the proportion of correctly matched spans in both predicted and ground-truth sets, providing a stricter evaluation than $\mathbf{F1}_M$ by requiring sufficient token overlap rather than partial inclusion.

Classification Metrics $\mathbf{F1}_{\text{Macro}}$ and $\mathbf{F1}_{\text{Micro}}$. These metrics evaluate hallucination classification across the C predefined hallucination types. For each type k , we count the number of true positives (TP_k), false positives (FP_k), and false negatives (FN_k) based on whether the predicted type matches the ground-truth type. $\mathbf{F1}_{\text{Macro}}$ computes F1 for each hallucination type independently and averages over all C classes, treating each type equally regardless of frequency, thus better reflecting balanced performance across rare and common types. $\mathbf{F1}_{\text{Micro}}$ computes F1 from globally aggregated TP, FP, and FN counts, and is more influenced by frequent classes, reflecting overall classification accuracy. Formal definitions are provided in Appendix.

Table 1: Results on the MHALO benchmark. $F1_M$ and $F1_{IoU}$ measure the quality of span-level hallucination detection, while IF denotes the format validity rate. The best results are highlighted in bold, and the second-best results are underlined.

Model	RLHF-V (500)			M-HalDetect (500)			Geo170K (500)			MathV360K (500)			Average		
	$F1_M$	$F1_{IoU}$	IF	$F1_M$	$F1_{IoU}$	IF	$F1_M$	$F1_{IoU}$	IF	$F1_M$	$F1_{IoU}$	IF	$F1_M$	$F1_{IoU}$	IF
Baselines															
Qwen3-VL-4B-Instruct	33.81	27.36	99.6	16.33	13.06	95.2	40.74	26.05	94.8	41.69	39.46	87.6	33.14	26.48	94.3
Qwen3-VL-8B-Instruct	40.30	34.62	99.6	18.61	13.69	99.0	37.40	24.46	86.8	51.12	48.93	89.0	36.86	30.43	93.6
Gemini-3-Flash	53.20	<u>43.26</u>	100.0	45.25	36.45	100.0	63.95	48.55	100.0	74.87	66.06	100.0	59.32	48.58	100.0
Gemini-3-Pro	50.71	40.68	100.0	48.37	<u>41.61</u>	100.0	63.14	46.96	100.0	72.20	62.13	100.0	58.61	47.84	100.0
GPT-4o	45.83	34.23	100.0	36.11	32.28	100.0	57.38	42.58	99.6	69.49	59.53	99.8	52.20	42.16	99.8
GPT-5	51.26	41.32	100.0	40.31	31.80	100.0	53.19	38.08	100.0	63.53	49.52	100.0	52.07	40.18	100.0
HALODET-4B [†]	35.43	28.66	100.0	27.41	24.66	99.4	44.34	34.58	80.2	73.82	68.82	97.6	45.25	39.18	94.3
HALODET-8B [†]	37.94	30.52	99.8	20.34	18.92	100.0	35.13	26.43	83.4	66.61	61.36	98.2	40.00	34.31	95.3
Our Models															
HalluScope-4B	<u>54.63</u>	43.66	100.0	44.36	39.36	100.0	<u>68.37</u>	<u>64.14</u>	99.0	<u>77.77</u>	<u>73.11</u>	99.4	<u>61.28</u>	<u>55.07</u>	99.6
HalluScope-8B	55.17	43.66	100.0	<u>47.24</u>	45.31	99.8	74.65	67.05	99.4	79.06	74.26	99.0	64.03	57.57	99.6

[†] Since the model weights are not publicly available, we retrain HALODET following the original paper’s method and data, using the same base models as our approach.

Table 2: Main results on our hallucination classification benchmark. $F1_M$ and $F1_{IoU}$ evaluate span-level detection, while $F1_{Macro}$ and $F1_{Micro}$ evaluate type classification. The best results are highlighted in bold, and the second-best results are underlined.

Model	$F1_M$	$F1_{IoU}$	$F1_{Macro}$	$F1_{Micro}$
Qwen3-VL-4B-Instruct	44.03	36.91	27.55	35.27
Qwen3-VL-8B-Instruct	56.47	49.27	33.73	40.28
Gemini-3-Flash	67.33	60.45	44.44	47.87
Gemini-3-Pro	67.55	61.17	<u>49.14</u>	53.14
GPT-4o	61.67	55.30	40.10	48.12
GPT-5	62.70	55.21	43.93	43.54
HalluScope-4B	<u>69.76</u>	<u>64.77</u>	48.46	<u>55.79</u>
HalluScope-8B	72.20	67.69	51.66	59.60

5.1.2 Benchmark. We evaluate our models on the fine-grained hallucination diagnosis task. As this task involves multiple stages, we assess model performance separately at different stages.

For the hallucination detection stage, we adopt the MHALO benchmark [4] for evaluation. MHALO is a benchmark designed for fine-grained hallucination detection and contains 2,000 samples drawn from four public subsets: RLHF-V [49], M-HalDetect [16], Geo170K [13], and MathV360K [36]. We evaluate the fine-grained hallucination detection capability of each model on every subset and compare the results with a series of representative MLLMs as well as the specialized HALODET baselines. Despite sharing source datasets, we ensure no sample overlap between our training data and the MHALO test set.

For the hallucination classification stage, since there is currently no public benchmark dedicated to fine-grained hallucination classification, we further construct a benchmark containing 733 samples.

This benchmark is built from datasets including MMBench [28], MMStar [5], OCRBench [29], RealWorldQA [45], Geo170K [13], MathV360K [36], M-HalDetect [16], and RLHF-V [49], and is designed to evaluate model performance across multiple task categories, including general perception, mathematical reasoning, OCR, spatial reasoning, and hallucination detection. The detailed composition of this benchmark is provided in Appendix.

5.1.3 Baselines. We compare our models with several strong baselines. Since HalluScope-4B and HalluScope-8B are obtained by post-training Qwen3-VL-4B-Instruct and Qwen3-VL-8B-Instruct [2], respectively, we include these two base models as fundamental baselines. We further compare our models against representative MLLMs, including Gemini-3-Flash, Gemini-3-Pro [39], GPT-4o [19], and GPT-5 [37]. We also include HALODET-4B and HALODET-8B [4] as specialized baselines for fine-grained hallucination detection. Since their model weights are not publicly available, we retrain both models following the original method and data, using the same base models as our approach. Since HALODET is designed solely for hallucination detection without type classification, it is not included in the classification benchmark evaluation.

5.1.4 Implementation Details. We use Qwen3-VL-4B-Instruct and Qwen3-VL-8B-Instruct as base models. The SFT stage trains for 1 epoch, and the RL stage trains for 250 steps with $\lambda = 0.3$. Full hyperparameters, hardware configuration, and ablation on λ are provided in Appendix.

5.2 Main Results

This section presents the evaluation results of HalluScope-4B and HalluScope-8B. Table 1 and Table 2 report the performance of the models on the MHALO benchmark and our hallucination classification benchmark, respectively. From these results, we highlight the following findings:

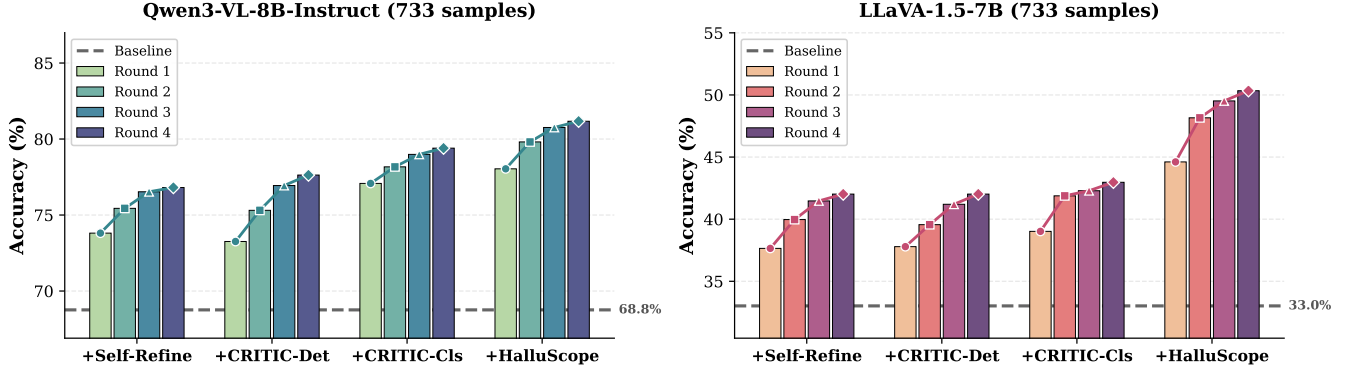


Figure 3: Accuracy (%) over four feedback rounds with different diagnostic granularities on two target models.

HalluScope-4B and HalluScope-8B achieve state-of-the-art performance on the MHALO benchmark. As shown in Table 1, both HalluScope models achieve state-of-the-art results on the MHALO benchmark. HalluScope-8B obtains the highest average $F1_M$ of 64.03 and $F1_{IoU}$ of 57.57, outperforming the strongest baseline Gemini-3-Flash by 4.71 and 8.99 points, respectively. HalluScope-4B also surpasses all baselines with an average $F1_M$ of 61.28 and $F1_{IoU}$ of 55.07. Compared with the specialized baselines HALODET-4B and HALODET-8B, our models show substantial improvements, indicating that our data generation pipeline and multi-granular reward function are more effective than the original HALODET training strategy. Furthermore, both models outperform their respective base models (Qwen3-VL-4B/8B-Instruct) by approximately 27 points on average, confirming the effectiveness of our post-training approach. Notably, since our training data is annotated by Gemini-3-Pro, one may worry that the models simply replicate Gemini’s annotation patterns. However, the fact that our models consistently outperform Gemini-3-Pro on the MHALO benchmark, which is an external benchmark not constructed by our pipeline, suggests that our models acquire genuine diagnostic capabilities beyond the annotation source.

Our models achieve state-of-the-art performance on the hallucination classification benchmark. Both HalluScope-4B and HalluScope-8B outperform all baseline models across all four evaluation metrics. HalluScope-8B achieves the best results with $F1_{Macro}$ of 51.66 and $F1_{Micro}$ of 59.60, while HalluScope-4B also surpasses the strongest baseline Gemini-3-Pro in $F1_M$, $F1_{IoU}$, and $F1_{Micro}$. These results demonstrate that our approach effectively improves both hallucination detection and type classification through the combination of fine-grained diagnostic data and multi-granular reward optimization.

5.3 Out-of-Distribution Generalization

Since our classification benchmark is produced by the same data generation pipeline as the training set, one may question whether the classification improvements generalize beyond this distribution. To address this concern, we further evaluate on HalLoc [32], an independent fine-grained hallucination classification benchmark that HalluScope has never seen during training. We map our 12 hallucination types to HalLoc’s three categories (object, attribute,

Table 3: Out-of-distribution classification results on the HalLoc Caption benchmark (%). Det denotes Detection Coverage and E2E denotes end-to-end accuracy (jointly detected and correctly classified). The best results are highlighted in bold.

Model	Object		Attribute		Relationship		Overall	
	Det	E2E	Det	E2E	Det	E2E	Det	E2E
Qwen3-VL-8B-Instruct	65.1	64.6	42.1	24.3	14.6	5.4	44.2	35.2
GPT-5.5	85.4	84.4	73.2	56.0	64.2	31.5	75.7	60.9
Gemini-3-Pro	81.5	81.1	68.4	50.3	51.9	46.5	69.4	61.3
HalluScope-8B	92.1	90.7	86.8	57.8	87.7	59.8	89.1	71.0

Table 4: Ablation on MHALO Benchmark (HalluScope-8B): Ours vs. Ours w/o cls.

Subset	Ours		Ours w/o cls	
	$F1_M$	$F1_{IoU}$	$F1_M$	$F1_{IoU}$
RLHF-V	55.17	43.66	53.34	44.25
M-HalDetect	47.24	45.31	48.50	45.03
Geo170K	74.65	67.05	65.22	58.10
MathV360K	79.06	74.26	78.07	73.82
Average	64.03	57.57	61.28	55.30

and relationship) and evaluate on the HalLoc Caption benchmark (3,079 samples) using two span-level metrics: Detection Coverage (Det) and end-to-end accuracy (E2E), where E2E requires a span to be both correctly detected and correctly classified. Using identical prompts across all models, Table 3 shows that HalluScope-8B achieves the best E2E accuracy across all three categories, reaching an overall E2E of **71.0** and outperforming both GPT-5.5 (60.9) and Gemini-3-Pro (61.3). This confirms that our model acquires genuine fine-grained classification ability that generalizes to out-of-distribution data, rather than merely fitting the annotation patterns of our pipeline.

Table 5: Ablation on Our Benchmark (HalluScope-8B): Ours vs. Ours w/o det.

Subset	Ours		Ours w/o det	
	F1 _{Macro}	F1 _{Micro}	F1 _{Macro}	F1 _{Micro}
MMBench	59.16	68.97	48.14	52.22
MMStar	58.26	62.15	46.81	46.05
OCRBench	81.60	81.02	45.68	56.18
RealWorldQA	54.82	69.21	45.31	47.23
Geo170K	28.53	41.59	17.20	23.80
MathV360K	50.00	59.11	38.75	43.90
M-HalDetect	39.05	63.69	21.46	37.18
RLHF-V	51.89	61.46	39.02	33.45
Average	51.66	59.60	38.17	40.54

5.4 Ablation Study

To validate the effectiveness of our multi-granular joint reward function, we conduct ablation experiments by training two variant models: one using only the detection reward $r_{\text{det}}(y_k)$ and the other using only the classification reward $r_{\text{cls}}(y_k)$. As shown in Table 4, removing the classification reward leads to a decline in detection performance on the MHALO benchmark, with average F1_M dropping from 64.03 to 61.28 and F1_{IOU} from 57.57 to 55.30. This suggests that the classification objective provides complementary supervision that also benefits span-level detection. As shown in Table 5, removing the detection reward causes a more substantial degradation in classification performance on our benchmark, with average F1_{Macro} dropping from 51.66 to 38.17 and F1_{Micro} from 59.60 to 40.54. This indicates that accurate span detection is a prerequisite for effective type classification. These results confirm that jointly optimizing detection and classification through the multi-granular reward function yields better performance than optimizing either objective alone.

5.5 Diagnosis-driven Feedback Experiments

After evaluating the performance of our models, a natural question arises: what practical value does it provide? The most direct answer is that it can offer diagnostic explanations for incorrect answers, enabling self-improvement of the target model. To this end, we explore a diagnosis-driven multi-round feedback experiment on two models: Qwen3-VL-8B-Instruct and LLaVA-1.5-7B. We evaluate on the 733 samples from our classification benchmark, where each target model first generates an initial response; HalluScope-8B then analyzes this response, identifies hallucinated spans along with their types, and provides explanations for the target model to regenerate its answer. This process can be iterated for multiple rounds. Detailed experimental setup is provided in Appendix.

To study how diagnostic granularity affects mitigation effectiveness, we compare four feedback modes. **Self-Refine** [30] serves as a baseline for pure self-correction, where the target model is simply informed that its previous answer was incorrect and prompted to retry without any external diagnosis. Building on the verify-then-correct paradigm of **CRITIC** [15], we design two diagnosis-driven baselines with increasing granularity: **CRITIC-Det** provides

Table 6: Diagnosis-driven feedback results on the AMBER benchmark after four rounds. The best results are highlighted in bold.

Method	Qwen3-VL-8B-Instruct				LLaVA-1.5-7B			
	CHAIR↓	Cover↑	Hal↓	Cog↓	CHAIR↓	Cover↑	Hal↓	Cog↓
Baseline	7.5	73.7	54.2	2.7	7.5	50.7	33.3	3.9
Self-Refine	8.8	75.0	64.4	4.0	7.5	50.7	33.3	3.9
CRITIC-Det	7.6	72.8	53.1	2.9	6.6	50.2	30.4	3.3
CRITIC-Cls	7.6	72.9	54.2	3.0	6.2	50.0	28.3	3.1
HalluScope	7.4	73.4	52.9	2.7	4.2	50.8	20.5	2.0

detection-level feedback by marking hallucinated spans and generating structured correction instructions without type classification; **CRITIC-Cls** further incorporates type annotations (e.g., Object, Calculation_Error) into both the span marking and correction instructions, informing the target model not only what is wrong but also what type of error it is. Finally, **HalluScope** uses HalluScope-8B to directly detect hallucinated spans, classify their types, and generate interpretable explanations, informing the target model what is wrong, why it is wrong, and how it may be corrected.

As shown in Figure 3, increasing diagnostic granularity consistently leads to larger accuracy gains across feedback rounds. On Qwen3-VL-8B-Instruct with an initial accuracy of 68.76%, HalluScope raises the accuracy to 81.17% after 4 rounds, substantially outperforming CRITIC-Cls at 79.40%, CRITIC-Det at 77.63%, and Self-Refine at 76.81%. The same trend holds on LLaVA-1.5-7B, where HalluScope achieves 50.34% from an initial 33.02%. Note that HalluScope-8B shares the same base model as Qwen3-VL-8B-Instruct, which may introduce an advantage in diagnosis. To control for this confound, we additionally evaluate on LLaVA-1.5-7B, which uses a different architecture and base model. The consistent performance ordering across both models confirms that external diagnostic feedback is essential for reliable self-correction, and that increasing diagnostic granularity progressively enhances hallucination mitigation regardless of the target model’s architecture.

To further verify that diagnosis-driven feedback generalizes beyond our own benchmark, we additionally evaluate on AMBER [40], an independent LLM-free hallucination benchmark containing 1,004 images. We adopt its rule-based metrics: CHAIR (object hallucination rate), Cover (object coverage), Hal (sentence-level hallucination rate), and Cog (cognition-level hallucination rate), and apply four rounds of iterative feedback to the same two target models. As shown in Table 6, HalluScope feedback consistently reduces hallucination on both models, most notably lowering CHAIR from 7.5 to 4.2 and Hal from 33.3 to 20.5 on LLaVA-1.5-7B, while maintaining or improving object coverage. In contrast, Self-Refine may even increase hallucination (Hal rising from 54.2 to 64.4 on Qwen3-VL-8B-Instruct) due to unguided regeneration. These results confirm that our fine-grained diagnostic feedback generalizes to an external benchmark and reliably improves generation quality, whereas coarser or diagnosis-free baselines provide limited or even negative effects.

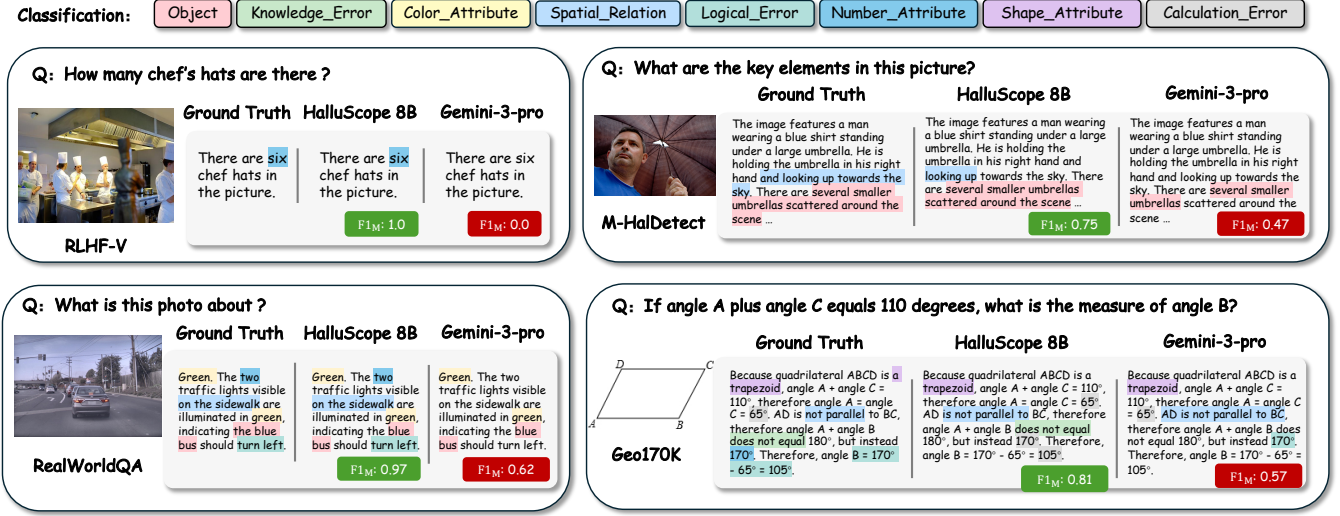


Figure 4: Qualitative comparison between HalluScope-8B and Gemini-3-Pro on four representative samples. Colored spans indicate detected hallucinations with type-specific annotations. F1_M scores are shown for each prediction.

5.6 Case Study

Figure 4 presents four representative examples comparing HalluScope-8B with Gemini-3-Pro across different datasets and hallucination types. In the RLHF-V example, HalluScope-8B precisely identifies the numerical hallucination “six” and achieves a perfect F1_M of 1.0, while Gemini-3-Pro fails to detect any hallucination (F1_M: 0.0). For the M-HalDetect sample containing multiple hallucination types (Object, Spatial_Relation), HalluScope-8B locates most hallucinated spans with F1_M of 0.75, whereas Gemini-3-Pro only achieves 0.47 with less precise boundaries. The RealWorldQA case demonstrates that HalluScope-8B can accurately detect subtle perceptual errors in real-world scenes (F1_M: 0.97 vs. 0.62). The Geo170K example involves reasoning hallucinations (Logical_Error, Calculation_Error, Knowledge_Error) in a geometry problem, where HalluScope-8B achieves F1_M of 0.81 versus 0.57 for Gemini-3-Pro, demonstrating stronger capability in identifying logical and computational errors. These cases illustrate that HalluScope-8B consistently produces more accurate span-level detection and more precise type classification across both perceptual and reasoning hallucinations.

6 CONCLUSION

In this paper, we introduce fine-grained hallucination diagnosis, a unified task that encompasses hallucination detection, classification, and interpretable explanation generation for MLLMs. We develop an automated data generation pipeline to construct HalluScope-30K, a large-scale dataset with nearly 30,000 diagnostic annotations across eight datasets and five task categories, and design a multi-granular joint reward function to train two diagnosis models, HalluScope-4B and HalluScope-8B. Although the current pipeline relies on a closed-source model for hallucination injection and annotation, which we plan to address with open-source alternatives in future work, experiments demonstrate that our models achieve state-of-the-art performance on both the MHALO detection benchmark and

our classification benchmark. Beyond detection accuracy, we find that detection and classification are mutually beneficial under joint optimization, and that increasing diagnostic granularity progressively improves hallucination mitigation, validating the practical value of fine-grained diagnosis.

References

- [1] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. 2015. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*. 2425–2433.
- [2] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. 2025. Qwen3-v1 technical report. *arXiv preprint arXiv:2511.21631* (2025).
- [3] Zechen Bai, Pichao Wang, Tianjun Xiao, Tong He, Zongbo Han, Zheng Zhang, and Mike Zheng Shou. 2024. Hallucination of multimodal large language models: A survey. *arXiv preprint arXiv:2404.18930* (2024).
- [4] Yishuo Cai, Renjie Gu, Jiaxu Li, Xuancheng Huang, Junzhe Chen, Xiaotao Gu, and Minlie Huang. 2025. MHALO: Evaluating MLLMs as Fine-grained Hallucination Detectors. In *Findings of the Association for Computational Linguistics: ACL 2025*. 9197–9222.
- [5] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, et al. 2024. Are we on the right way for evaluating large vision-language models? *Advances in Neural Information Processing Systems* 37 (2024), 27056–27087.
- [6] Xinlei Chen, Hao Fang, Tsung-Yi Lin, Ramakrishna Vedantam, Saurabh Gupta, Piotr Dollár, and C Lawrence Zitnick. 2015. Microsoft coco captions: Data collection and evaluation server. *arXiv preprint arXiv:1504.00325* (2015).
- [7] Xiang Chen, Chenxi Wang, Yida Xue, Ningyu Zhang, Xiaoyan Yang, Qiang Li, Yue Shen, Lei Liang, Jinjie Gu, and Huajun Chen. 2024. Unified hallucination detection for multimodal large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3235–3252.
- [8] Zhiyuan Chen, Yuecong Min, Jie Zhang, Bei Yan, Jiahao Wang, Xiaozhen Wang, and Shiguang Shan. 2026. A survey of multimodal hallucination evaluation and detection. *International Journal of Computer Vision* 134, 3 (2026), 131.
- [9] Yun-Wei Chu, Kai Zhang, Christopher Malon, and Martin Renqiang Min. 2025. Reducing hallucinations of medical multimodal large language models with visual retrieval-augmented generation. *arXiv preprint arXiv:2502.15040* (2025).
- [10] Laura Fieback, Nishilkumar Balar, Jakob Spiegelberg, and Hanno Gottschalk. 2025. Efficient Contrastive Decoding with Probabilistic Hallucination Detection-Mitigating Hallucinations in Large Vision Language Models. *arXiv preprint arXiv:2504.12137* (2025).
- [11] Laura Fieback, Jakob Spiegelberg, and Hanno Gottschalk. 2024. Metatoken: Detecting hallucination in image descriptions by meta classification. *arXiv preprint arXiv:2405.19186* (2024).
- [12] Deqing Fu, Tong Xiao, Rui Wang, Wang Zhu, Pengchuan Zhang, Guan Pang, Robin Jia, and Lawrence Chen. 2024. Tldr: Token-level detective reward model for large vision language models. *arXiv preprint arXiv:2410.04734* (2024).
- [13] Jiahui Gao, Renjie Pi, Jipeng Zhang, Jiacheng Ye, Wanjuan Zhong, Yufei Wang, Lanqing Hong, Jianhua Han, Hang Xu, Zhenguo Li, et al. 2023. G-llava: Solving geometric problem with multi-modal large language model. *arXiv preprint arXiv:2312.11370* (2023).
- [14] Team Glm, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, et al. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793* (2024).
- [15] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2023. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738* (2023).
- [16] Anisha Gunjal, Jihan Yin, and Erhan Bas. 2024. Detecting and preventing hallucinations in large vision language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18135–18143.
- [17] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirog Ma, Xiao Bi, et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (2025), 633–638.
- [18] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [19] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [20] Nazamin Jafari, James Allan, and Sheikh Muhammad Sarwar. 2024. Target span detection for implicit harmful content. In *Proceedings of the 2024 ACM SIGIR International Conference on Theory of Information Retrieval*. 117–122.
- [21] Jiayi Kuang, Ying Shen, Jingyou Xie, Haoqiao Luo, Zhe Xu, Ronghao Li, Yinghui Li, Xianfeng Cheng, Xika Lin, and Yu Han. 2025. Natural language understanding and inference with mllm in visual question answering: A survey. *Comput. Surveys* 57, 8 (2025), 1–36.
- [22] Harold W Kuhn. 1955. The Hungarian method for the assignment problem. *Naval research logistics quarterly* 2, 1-2 (1955), 83–97.
- [23] Sicong Leng, Hang Zhang, Guanzheng Chen, Xin Li, Shijian Lu, Chunyan Miao, and Lidong Bing. 2024. Mitigating object hallucinations in large vision-language models through visual contrastive decoding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13872–13882.
- [24] Chunyuan Li, Cliff Wong, Sheng Zhang, Naoto Usuyama, Haotian Liu, Jianwei Yang, Tristan Naumann, Hoifung Poon, and Jianfeng Gao. 2023. Llava-med: Training a large language-and-vision assistant for biomedicine in one day. *Advances in Neural Information Processing Systems* 36 (2023), 28541–28564.
- [25] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*. PMLR, 19730–19742.
- [26] Hanchao Liu, Wenyuan Xue, Yifei Chen, Dapeng Chen, Xiutian Zhao, Ke Wang, Liping Hou, Rongjun Li, and Wei Peng. 2024. A survey on hallucination in large vision-language models. *arXiv preprint arXiv:2402.00253* (2024).
- [27] Sheng Liu, Haotian Ye, and James Zou. 2025. Reducing hallucinations in large vision-language models via latent space steering. In *The Thirteenth International Conference on Learning Representations*.
- [28] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, et al. 2024. Mmbench: Is your multi-modal model an all-around player?. In *European conference on computer vision*. Springer, 216–233.
- [29] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xu-Cheng Yin, Cheng-Lin Liu, Lianwen Jin, and Xiang Bai. 2024. Ocrbench: on the hidden mystery of ocr in large multimodal models. *Science China Information Sciences* 67, 12 (2024), 220102.
- [30] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.
- [31] Abhika Mishra, Akari Asai, Vidhisha Balachandran, Yizhong Wang, Graham Neubig, Yulia Tsvetkov, and Hannaneh Hajishirzi. 2024. Fine-grained hallucination detection and editing for language models. *arXiv preprint arXiv:2401.06855* (2024).
- [32] Eunhyu Park, Minyeong Kim, and Gunhee Kim. 2025. Halloc: Token-level localization of hallucinations for vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 29893–29903.
- [33] Zhiqiang Peng, Wenhui Wang, Li Dong, Yaru Hao, Shaohan Huang, Shuming Ma, and Furu Wei. 2023. Kosmos-2: Grounding multimodal large language models to the world. *arXiv preprint arXiv:2306.14824* (2023).
- [34] Pranab Sahoo, Prabhash Meharia, Akash Ghosh, Sriparna Saha, Vinija Jain, and Aman Chadha. 2024. A comprehensive survey of hallucination in large language, image, video and audio foundation models. *Findings of the Association for Computational Linguistics: EMNLP 2024* (2024), 11709–11724.
- [35] Pritish Sahu, Karan Sikka, and Ajay Divakaran. 2024. Pelican: Correcting hallucination in vision-llms via claim decomposition and program of thought verification. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 8228–8248.
- [36] Wenhao Shi, Zhiqiang Hu, Yi Bin, Junhua Liu, Yang Yang, See Kiong Ng, Lidong Bing, and Roy Ka-Wei Lee. 2024. Math-llava: Bootstrapping mathematical reasoning for multimodal large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 4663–4680.
- [37] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267* (2025).
- [38] Zhiqing Sun, Sheng Shen, Shengcao Cao, Haotian Liu, Chunyuan Li, Yikang Shen, Chuang Gan, Liangyan Gui, Yu-Xiong Wang, Yiming Yang, et al. 2024. Aligning large multimodal models with factually augmented rlhf. In *Findings of the Association for Computational Linguistics: ACL 2024*. 13088–13110.
- [39] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [40] Junyang Wang, Yuhang Wang, Guohai Xu, Jing Zhang, Yukai Gu, Haitao Jia, Jiaqi Wang, Haiyang Xu, Ming Yan, Ji Zhang, et al. 2023. Amber: An llm-free multi-dimensional benchmark for mllms hallucination evaluation. *arXiv preprint arXiv:2311.07397* (2023).
- [41] Junyang Wang, Yiyang Zhou, Guohai Xu, Pengcheng Shi, Chenlin Zhao, Haiyang Xu, Qinghao Ye, Ming Yan, Ji Zhang, Jihua Zhu, et al. 2023. Evaluation and analysis of hallucination in large vision-language models. *arXiv preprint arXiv:2308.15126* (2023).
- [42] Xintong Wang, Jingheng Pan, Liang Ding, and Chris Biemann. 2024. Mitigating hallucinations in large vision-language models with instruction contrastive decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*. 15840–15853.
- [43] Yiqi Wang, Wentao Chen, Xiaotian Han, Xudong Lin, Haiteng Zhao, Yongfei Liu, Bohan Zhai, Jianbo Yuan, Quanzeng You, and Hongxia Yang. 2024. Exploring the reasoning abilities of multimodal large language models (mllms): A comprehensive survey on emerging trends in multimodal reasoning. *arXiv preprint arXiv:2401.06805* (2024).

- [44] Spencer Whitehead, Jacob Phillips, and Sean Hendryx. 2024. Pre-Training Multimodal Hallucination Detectors with Corrupted Grounding Data. *arXiv preprint arXiv:2409.00238* (2024).
- [45] xAI. 2024. Grok-1.5V: Multimodal Understanding with Grok. <https://x.ai/blog/grok-1.5v>
- [46] Wenyi Xiao, Ziwei Huang, Leilei Gan, Wanggui He, Haoyuan Li, Zhelun Yu, Fangxun Shu, Hao Jiang, and Linchao Zhu. 2025. Detecting and mitigating hallucination in large vision language models via fine-grained ai feedback. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 25543–25551.
- [47] Le Yang, Ziwei Zheng, Boxu Chen, Zhengyu Zhao, Chenhao Lin, and Chao Shen. 2025. Nullu: Mitigating object hallucinations in large vision-language models via halluspace projection. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 14635–14645.
- [48] Shukang Yin, Chaoyou Fu, Sirui Zhao, Tong Xu, Hao Wang, Dianbo Sui, Yunhang Shen, Ke Li, Xing Sun, and Enhong Chen. 2024. Woodpecker: Hallucination correction for multimodal large language models. *Science China Information Sciences* 67, 12 (2024), 220105.
- [49] Tianyu Yu, Yuan Yao, Haoye Zhang, Taiwen He, Yifeng Han, Ganqu Cui, Jinyi Hu, Zhiyuan Liu, Hai-Tao Zheng, Maosong Sun, et al. 2024. RLHF-v: Towards trustworthy mlms via behavior alignment from fine-grained correctional human feedback. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13807–13816.
- [50] Bohan Zhai, Shijia Yang, Chenfeng Xu, Sheng Shen, Kurt Keutzer, Chunyuan Li, and Manling Li. 2023. HallE-Control: controlling object hallucination in large multimodal models. *arXiv preprint arXiv:2310.01779* (2023).
- [51] Kaichen Zhang, Keming Wu, Zuhao Yang, Bo Li, Kairui Hu, Bin Wang, Ziwei Liu, Xingxuan Li, and Lidong Bing. 2025. OpenMMReasoner: Pushing the Frontiers for Multimodal Reasoning with an Open and General Recipe. *arXiv:2511.16334* [cs.AI] <https://arxiv.org/abs/2511.16334>
- [52] Ruiyang Zhang, Hu Zhang, and Zhedong Zheng. 2024. VL-uncertainty: Detecting hallucination in large vision-language model via uncertainty estimation. *arXiv preprint arXiv:2411.11919* (2024).
- [53] Yudong Zhang, Ruobing Xie, Xingwu Sun, Yiqing Huang, Jiansheng Chen, Zhanhui Kang, Di Wang, and Yu Wang. 2025. Dhcp: Detecting hallucinations by cross-modal attention pattern in large vision-language models. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 3555–3564.
- [54] Zhiyuan Zhao, Bin Wang, Linke Ouyang, Xiaoyi Dong, Jiaqi Wang, and Conghui He. 2023. Beyond hallucinations: Enhancing lvlms through hallucination-aware direct preference optimization. *arXiv preprint arXiv:2311.16839* (2023).
- [55] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592* (2023).
- [56] Lanyun Zhu, Deyi Ji, Tianrun Chen, Peng Xu, Jieping Ye, and Jun Liu. 2025. Ibd: Alleviating hallucinations in large vision-language models via image-biased decoding. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 1624–1633.
- [57] Geigh Zollicoffer, Minh Vu, and Manish Bhattarai. 2025. MTRE: Multi-Token Reliability Estimation for Hallucination Detection in VLMs. *arXiv preprint arXiv:2505.11741* (2025).

A Dataset Details

A.1 Dataset Composition

Table 7 summarizes the composition of HalluScope-30K. The dataset is split into a training set of 29,747 samples and a test set of 733 samples. Both sets cover all eight source datasets across five task categories, ensuring that the test set evaluates the full spectrum of hallucination types. The eight source datasets are:

- **MMBench** [28]: A multi-discipline visual understanding benchmark covering perception, reasoning, and knowledge across multiple languages.
- **MMStar** [5]: A curated multimodal benchmark emphasizing vision-critical questions that resist text-only shortcuts.
- **MathV360K** [36]: A large-scale mathematical visual QA collection spanning 24 sub-domains including geometry, algebra, and statistics.
- **Geo170K** [13]: A geometry-focused dataset with diagram-based reasoning questions from Geo3K and GeoQA+.
- **OCRBench** [29]: An OCR evaluation benchmark covering text recognition, scene text understanding, and document analysis.
- **RealWorldQA** [45]: Real-world spatial understanding questions requiring reasoning about object positions, orientations, and environments.
- **M-HalDetect** [16]: An image captioning and VQA dataset with COCO images, originally designed for multimodal hallucination detection.
- **RLHF-V** [49]: A visual QA dataset with human preference annotations; we use the chosen answer as ground truth.

Table 7: Composition of HalluScope-30K by source dataset and task category.

Task Category	Source Dataset	Train	Test	Total
General Perception	MMBench	5,372	113	5,485
	MMStar	726	58	784
Mathematical Reasoning	MathV360K	6,371	123	6,494
	Geo170K	3,899	123	4,022
OCR	OCRBench	735	44	779
Spatial Reasoning	RealWorldQA	526	42	568
Hallucination Detection	M-HalDetect	7,385	114	7,499
	RLHF-V	4,733	116	4,849
Total		29,747	733	30,480

A.2 Data Preprocessing

Each of the source datasets listed in Table 7 undergoes a standardized preprocessing pipeline before entering the hallucination label generation stage. The preprocessing consists of the following steps:

- (1) **Format unification.** Raw datasets differ in schema and annotation style. We convert all samples into a unified format with fields: image, question, answer, question type (MCQ or open-ended), and source metadata.

- (2) **Subset sampling.** For large-scale datasets (MathV360K and Geo170K), we use randomly sampled subsets to maintain a balanced composition across task categories.

- (3) **Deduplication.** We construct a deduplication key from the concatenation of question, choices, and answer, and remove duplicate entries keeping the first occurrence.

After preprocessing, all samples enter the hallucination label generation pipeline. The final HalluScope-30K is obtained after multi-stage quality control (Appendix C).

A.3 Injection vs. Annotation Ratio

As described in the main text, HalluScope employs two complementary label generation strategies based on model response correctness: *hallucination injection* for correctly answered samples and *hallucination annotation* for incorrectly answered ones. Table 8 reports the ratio of samples processed through each pathway.

Table 8: Ratio of hallucination injection (correct answers) vs. annotation (incorrect answers) by task category.

Task Category	Injection	Annotation	Inject. %
General Perception	6,806	1,640	80.6%
Math. Reasoning	10,072	5,156	66.1%
OCR	848	134	86.4%
Spatial Reasoning	494	236	67.7%
Halluc. Detection	0	12,118	0.0%
Total	18,220	19,284	48.6%

The ratio varies substantially across task categories. OCR and general perception datasets exhibit high injection rates of 86.4% and 80.6%, respectively, because the base model Qwen3-VL-8B-Instruct answers most of these questions correctly. Mathematical reasoning has a moderate injection rate of 66.1% due to the model’s lower accuracy on geometry and math problems. Hallucination detection datasets, namely M-HalDetect and RLHF-V, are entirely processed through the annotation pathway, as these datasets provide pre-existing model responses that already contain hallucinations.

A.4 Hallucination Type Distribution

Figure 5 illustrates the distribution of hallucination types in the training and test sets. Each sample may contain multiple hallucination spans of different types, so the span counts exceed the total number of samples.

The distribution exhibits notable class imbalance. *Object* is the most frequent type, accounting for 26.2% of all training spans, while *Query_Misunderstanding* is extremely underrepresented with only 373 training spans (0.27%) and 7 test spans. This imbalance directly explains the model’s low performance on these rare types (e.g., *Query_Misunderstanding*). On average, each sample contains 4.6 hallucination spans in both the training and test sets.

A.5 Benchmark Construction

The hallucination classification benchmark consists of 733 samples drawn from the same data generation pipeline as the training split,

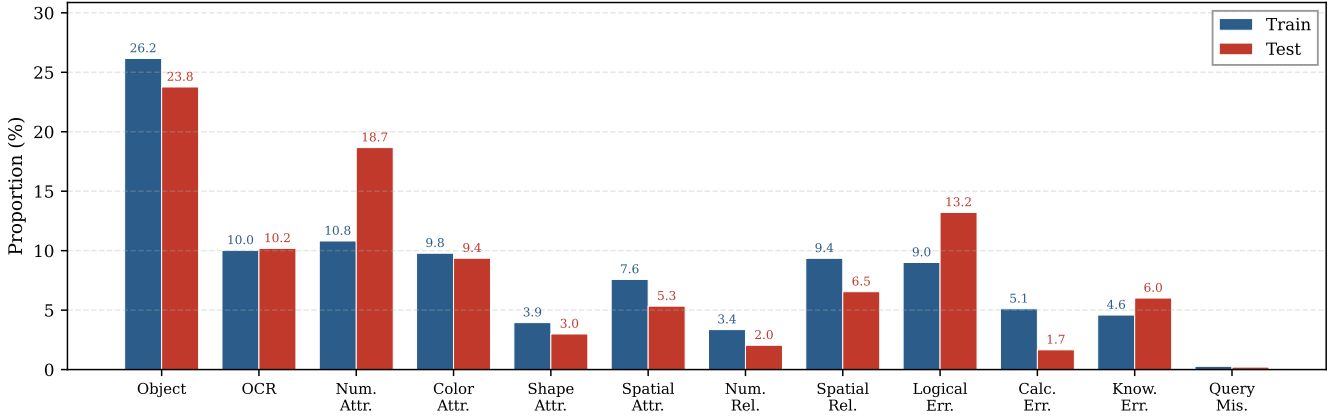


Figure 5: Proportion of each hallucination type in the training set (29,747 samples, 137,283 spans) and the test set (733 samples, 3,375 spans).

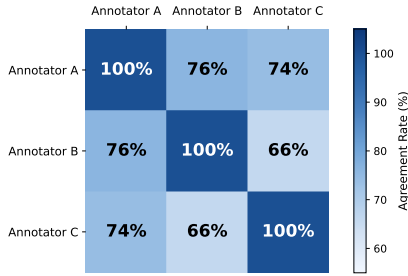


Figure 6: Pairwise agreement rates among three annotators on 100 randomly sampled test samples.

with strict deduplication to ensure no overlap between the two splits. The per-subset composition is shown in Table 7, covering all task categories. Each test sample undergoes the same quality control process as the training data, including text quality assessment and hallucination consistency verification.

To assess the annotation quality of the benchmark, three independent annotators each evaluated 100 randomly sampled test instances and judged whether the hallucination labels (span boundaries and type assignments) were correct. Note that this annotation task requires jointly verifying both the span boundary and the hallucination type from a fine-grained taxonomy of 12 categories, which is substantially more demanding than binary hallucination detection. Annotators A and B each reported an accuracy of 80.0%, while Annotator C reported 76.0%. The pairwise agreement rates are 76.0% (A–B), 74.0% (A–C), and 66.0% (B–C), as shown in Figure 6. The lower pairwise rates primarily reflect the inherent subjectivity in fine-grained type assignment rather than span detection errors: annotators typically agreed on whether a span is hallucinated but occasionally diverged on the assigned type (e.g., *Numerical_Attribute* vs. *Calculation_Error*). Under majority voting (at least two annotators agree), the accuracy reaches 86.0%, confirming the high quality of the benchmark annotations.

B Hallucination Taxonomy

We adopt the hallucination taxonomy proposed by MHALO [4], which organizes 12 fine-grained hallucination types into two high-level categories: *perception-level* and *reasoning-level*. Table 9 summarizes all types with their definitions.

Perception-level hallucinations arise from errors in visual understanding or information extraction, and can be directly verified by examining the image. They are organized into two sub-groups:

- **Content errors:** *Object*, which refers to misidentifying an object’s presence, absence, or category, and *OCR*, which refers to misreading visible text, numbers, or symbols.
- **Attribute errors:** *Numerical_Attribute* for wrong absolute count or measurement of a single entity, *Color_Attribute* for wrong color of a correctly identified object, *Shape_Attribute* for wrong geometric or structural form, and *Spatial_Attribute* for wrong position, orientation, or distance of a single object.

Reasoning-level hallucinations originate from incorrect inference, computation, or knowledge application, requiring cognitive processing beyond direct visual observation. They are organized into two sub-groups:

- **Relation errors:** *Numerical_Relation* for wrong quantitative comparison between multiple entities, and *Spatial_Relation* for wrong spatial arrangement between multiple objects. Although relation errors may involve visual content, they require comparative reasoning across multiple objects rather than direct perception of a single element.
- **Reasoning errors:** *Logical_Error* for flawed reasoning chains or invalid inferences, *Calculation_Error* for mistakes in arithmetic or mathematical computation, *Knowledge_Error* for incorrect application of external knowledge or domain-specific facts, and *Query_Misunderstanding* for responding to a different question than what was asked.

Table 9: Hallucination types and definitions used for hallucination diagnosis, adopted from the taxonomy of MHALO [4]. Types above the mid-rule are perception-level; types below are reasoning-level.

Type	Definition
Object	Errors in recognizing an object’s presence, absence, or category.
OCR	Failure to correctly read visible text, numbers, or symbols.
Numerical Attribute	Errors in identifying the absolute value of a single object or element.
Color Attribute	Misperception of the color of a correctly identified object or region.
Shape Attribute	Errors in describing an object’s geometric or structural shape.
Spatial Attribute	Errors in identifying a single object’s position, orientation, or distance.
Numerical Relation	Errors in reasoning about quantitative relations among objects or values.
Spatial Relation	Errors in reasoning about spatial relations among multiple objects.
Logical Error	Wrong conclusions caused by flawed logical inference.
Calculation Error	Mistakes in arithmetic or mathematical computation.
Knowledge Error	Incorrect use of external knowledge, common sense, or domain rules.
Query Misunderstanding	Responses caused by misunderstanding the query intent or constraints.

C Quality Control Details

C.1 Label Normalization

The label normalization stage maps non-standard hallucination type labels produced by the LLM annotator to the standardized set of 12 types. The pipeline applies over 80 rule-based mappings, including case normalization (e.g., “object” → “Object”), plural-to-singular conversion (e.g., “Calculation_Errors” → “Calculation_Error”), format unification for space-separated and CamelCase variants, and semantic synonym mapping (e.g., “Entity” → “Object”, “Factual_Error” → “Knowledge_Error”). If no rule matches, fuzzy matching is attempted by comparing labels after stripping underscores, spaces, and hyphens. Samples containing unmappable labels (e.g., “Unknown”, “None”) are discarded entirely.

C.2 Text Quality Assessment

We use LLM-as-Judge to assess the text quality of each generated sample. The judge evaluates three dimensions on a 1–5 scale:

- **Clarity** (1–5): Whether the response is easy to understand, with unambiguous language and clear expression of ideas.
- **Fluency** (1–5): Whether the response reads naturally, with proper grammar, word choice, and sentence flow.
- **Coherence** (1–5): Whether the response maintains logical consistency throughout, with smooth transitions between ideas and no contradictions.

A sample passes quality assessment only if it scores ≥ 4 on all three dimensions. Samples scoring below the threshold on any dimension are removed from the dataset.

C.3 Hallucination Consistency Verification

After label normalization and text quality assessment, we use LLM-as-Judge to verify the consistency of the generated hallucination labels. Specifically, for each sample, the judge model is provided with the original image, question, ground-truth answer, and the hallucination-annotated response, and is asked to evaluate two

aspects: (1) whether each tagged hallucination is semantically plausible as a realistic MLLM error, and (2) whether the surrounding text remains coherent after hallucination injection. In addition, rule-based checks are applied to ensure format correctness (proper XML tag closure, no nesting) and type validity (each type attribute belongs to the predefined 12-type set). Samples that fail any of these checks are removed from the final dataset.

D Implementation Details

D.1 Training Hyperparameters

All experiments are conducted on four servers, including two servers equipped with 2 NVIDIA H20Z GPUs (144 GB) and two servers equipped with 2 NVIDIA B20Z GPUs (180 GB). We use Qwen3-VL-4B-Instruct and Qwen3-VL-8B-Instruct as the base models.

SFT stage. We apply LoRA fine-tuning with rank $r = 8$ and alpha $\alpha = 16$, targeting all linear layers with the vision tower frozen. Training runs for 1 epoch with a learning rate of 5×10^{-5} , a batch size of 8, and a maximum sequence length of 4096. After SFT, the LoRA adapters are merged into the base model.

RL stage. We apply GRPO with a group size $K = 5$ and full-parameter training (no LoRA) with the vision tower unfrozen. The learning rate is 1×10^{-6} , global batch size is 32, and training runs for 250 steps. The KL coefficient is 0.01 and the reward weight λ is set to 0.3. The maximum prompt length is 4096 and maximum response length is 2048. During training rollout, temperature is 1.0 and top- p is 1.0; during validation, temperature is 0.6 and top- p is 0.95. We use FSDP with CPU offloading and vLLM for rollout generation. The RL framework is based on EasyR1 (built on vLLM).

D.2 Mitigation Experiment Setup

We provide additional details on the mitigation experiment. The initial accuracy before feedback is 68.76% (504/733) for Qwen3-VL-8B-Instruct and 33.02% (242/733) for LLaVA-1.5-7B. All models use greedy decoding (temperature = 0) with a maximum generation

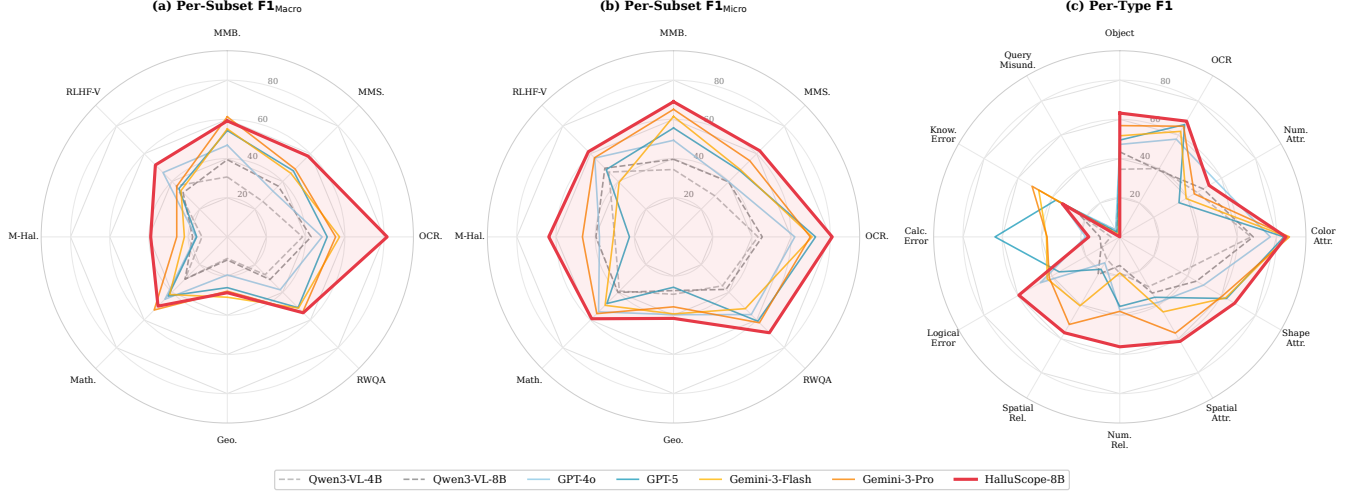


Figure 7: Per-subset and per-type classification results on our hallucination classification benchmark. (a) Per-subset $F1_{Macro}$ (b) Per-subset $F1_{Micro}$ (c) Per-Type F1 across 12 hallucination types.

Table 10: Effect of λ on detection and classification performance. Best results per metric are bolded.

λ	MHALO		Classification		
	Avg $F1_M$	Avg $F1_{IoU}$	$F1_M$	$F1_{Macro}$	$F1_{Micro}$
0.1	61.97	56.65	72.48	49.74	58.54
0.3	64.03	57.57	72.20	51.66	59.60
0.5	62.88	56.98	71.14	52.53	59.55
0.7	64.92	59.10	70.70	51.87	58.88

length of 2048 tokens. For free-form questions, correctness is determined by Gemini-3-Pro judge through semantic equivalence; for MCQs, the option letter is extracted and compared directly. Feedback prompt templates for all four modes are provided in Appendix H.3.

E Evaluation Metrics

$F1_{Macro}$:

$$F1_{Macro} = \frac{1}{C} \sum_{k=1}^C \frac{2TP_k}{2TP_k + FP_k + FN_k}. \quad (10)$$

This metric is insensitive to class frequency and better reflects balanced performance across hallucination types.

$F1_{Micro}$:

$$F1_{Micro} = \frac{2 \sum_{k=1}^C TP_k}{2 \sum_{k=1}^C TP_k + \sum_{k=1}^C FP_k + \sum_{k=1}^C FN_k}. \quad (11)$$

$F1_{Micro}$ is computed from globally aggregated statistics and is more influenced by frequent classes.

F Additional Experimental Results

Figure 7 presents the per-subset and per-type classification results on our hallucination classification benchmark.

Per-subset analysis. HalluScope-8B achieves the highest $F1_{Micro}$ on all eight subsets. The most pronounced advantage appears

Table 11: Multiplicative vs. additive detection reward composition. Best results per metric are bolded.

Composition	MHALO		Classification		
	Avg $F1_M$	Avg $F1_{IoU}$	$F1_M$	$F1_{Macro}$	$F1_{Micro}$
Multiplicative	64.03	57.57	72.20	51.66	59.60
Additive	62.70	57.36	71.13	51.99	59.46

on OCRBench ($F1_{Macro}$ =81.6 vs. 57.3 for Gemini-3-Flash) and M-HalDetect ($F1_{Micro}$ =63.7 vs. 46.5 for Gemini-3-Pro), indicating that our model particularly excels at OCR-related and open-ended hallucination classification. Geo170K remains the most challenging subset for all models ($F1_{Micro} < 42$), reflecting the difficulty of fine-grained type classification on geometry problems.

Per-type analysis. HalluScope-8B achieves the best F1 on 7 of 12 hallucination types. The largest gains appear on reasoning-level types: *Numerical_Relation* (56.1 vs. 38.0 for Gemini-3-Pro), *Logical_Error* (59.5 vs. 41.6), and *Spatial_Relation* (56.5 vs. 51.6). A notable weakness is *Calculation_Error* ($F1$ =15.8), where GPT-5 reaches 63.7, suggesting that numerical computation verification benefits more from the stronger reasoning capabilities of larger models. *Query_Misunderstanding* yields $F1$ =0.0 for both HalluScope-8B and Gemini-3-Pro, consistent with its extreme scarcity in the dataset (only 7 test spans, see Appendix A.4).

F.1 Explanation Quality Evaluation

Beyond detection and classification, HalluScope also generates natural language explanations that describe why a hallucination occurs and how it may be corrected. To directly assess the quality of these explanations, we randomly sample 50 diagnosed cases from our classification benchmark and invite three independent annotators to rate each explanation on a 1–5 scale along three dimensions: **Causal Accuracy (CA)**, whether the explanation correctly identifies the cause of the hallucination; **Correction Usefulness (CU)**,

whether it provides actionable correction guidance; and **Visual Grounding (VG)**, whether the reasoning is grounded in image evidence.

As shown in Table 12, the explanations receive an average score of 3.89, with particularly high Visual Grounding (4.61), indicating that the diagnostic reasoning is well grounded in the visual input. The inter-annotator agreement (pairwise agreement within one point) reaches 70.7% for CA, 82.0% for CU, and 86.0% for VG, indicating good consistency across all three dimensions. These results directly validate the quality of HalluScope’s diagnostic explanations.

Table 12: Human evaluation of explanation quality on 50 randomly sampled diagnosed cases. CA: Causal Accuracy; CU: Correction Usefulness; VG: Visual Grounding.

Annotator	CA↑	CU↑	VG↑	Avg↑
Annotator #1	3.90	3.62	4.42	3.98
Annotator #2	3.40	3.28	4.46	3.71
Annotator #3	3.72	3.24	4.96	3.97
Average	3.67	3.38	4.61	3.89

G Additional Ablation Studies

G.1 Reward Weight Sensitivity (λ)

As described in the main text, the RL stage optimizes a multi-granular joint reward that combines a span-level detection reward $r_{\text{det}}(y_k)$ with a type-level classification reward $r_{\text{cls}}(y_k)$. The hyperparameter λ controls the relative importance of these two objectives. A higher λ allocates more training signal toward correct type classification, while a lower λ prioritizes accurate span detection. The multi-granular joint reward is formulated as:

$$R(y_k) = \mathbb{I}_{\text{format}} ((1 - \lambda) r_{\text{det}}(y_k) + \lambda r_{\text{cls}}(y_k)) \quad (12)$$

We train models with $\lambda \in \{0.1, 0.3, 0.5, 0.7\}$ and report results on both the MHALO benchmark and our hallucination classification benchmark in Table 10.

Increasing λ allocates more training signal toward type classification. As shown in Table 10, $\lambda = 0.7$ achieves the highest MHALO detection scores with Avg $F1_M=64.92$ and Avg $F1_{\text{IoU}}=59.10$, likely because the classification reward r_{cls} also implicitly encourages accurate span detection through type-matched IoU. However, it degrades performance on our hallucination classification benchmark to $F1_M=70.70$. Conversely, $\lambda = 0.1$ maximizes $F1_M$ at 72.48 but yields the weakest MHALO detection at 61.97 and the lowest $F1_{\text{Macro}}$ of 49.74. We select $\lambda = 0.3$ as the default because it achieves competitive MHALO detection at 64.03 while maintaining strong classification performance across all three metrics: $F1_M=72.20$, $F1_{\text{Macro}}=51.66$, and $F1_{\text{Micro}}=59.60$, providing the best overall balance.

G.2 Detection Reward Composition: Multiplicative vs. Additive

We compare two composition modes for the detection reward: the multiplicative form $r_{\text{det}} = r_{\text{content}} \times r_{\text{count}}$ and an additive form $r_{\text{det}} = 0.5 \cdot r_{\text{content}} + 0.5 \cdot r_{\text{count}}$. Table 11 summarizes the results.

The multiplicative form outperforms the additive variant on most metrics, with a notable advantage on MHALO detection where $F1_M$ reaches 64.03 vs. 62.70 for the additive form. This is expected because the multiplicative composition requires both content matching and count accuracy to be high simultaneously, providing a stronger training signal that discourages partial matches.

H Prompt Templates

H.1 Data Generation Prompts

Our data generation pipeline uses four prompt-driven stages: Stage 1 generates model answers and judges correctness, Stage 2 applies hallucination label generation through injection or annotation, Stage 3 assesses linguistic quality, and Stage 4 verifies hallucination consistency. The full prompts are listed below.

Stage 1a: Model Answer Generation

You are an expert visual question answering assistant.

Question: {question}

Instructions: Analyze the image carefully and think step by step.

1. Generate a step-by-step reasoning process that naturally leads to the correct answer
2. Show your complete thought process and analysis
3. Make sure your reasoning is logical, coherent, and educational
4. Finally, conclude with the provided correct answer

Important:

- Be thorough in your reasoning and analysis
 - Consider all relevant information from the question
 - Your reasoning should naturally lead to the answer
 - Make the reasoning process natural and instructive
- Please provide your reasoning and answer.

Stage 1b: Answer Correctness Judge

Determine if the model’s answer is semantically correct compared to the ground truth.

- For numerical answers: exact match required
- For descriptive answers: semantic equivalence is sufficient

Question: {question}

Ground Truth: {ground_truth}

Model Answer: {model_answer}

Instructions: Determine if the Model Answer is consistent with the Ground Truth Answer. Consider:

1. The core meaning/result should be the same
2. Numerical answers should match exactly
3. For multiple choice, the selected option should match

Respond with ONLY one of the following: "CORRECT" or "INCORRECT"

Stage 2a: Hallucination Injection (for correct samples)
You are an expert at injecting hallucinations into visual question answering data. Task: Inject 3–5 different types of hallucinations into the given correct model answer to create a plausible but incorrect response. Each hallucination tag should cover a natural semantic unit — use short phrases (1–3 words) when possible, but longer spans are acceptable when the error is a coherent phrase. Hallucination Types: <i>[12 types with definitions and boundary rules]</i> Dataset Category: {dataset_category} Priority Types (use 2–3): {primary_types} Secondary Types (use 1–2): {secondary_types} Injection Strategies: Apply one or more of the following strategies to inject hallucinations naturally: (1) Entity Replacement: Replace correct entities with similar but incorrect ones. (2) Object Attribute Modification: Change attributes while keeping the object. (3) Object Relationship Modification: Change relationships between multiple objects. (4) Numerical Perturbation: Modify numerical values. (5) Precision-to-Vagueness Conversion: Replace precise info with misleading vague expressions. (6) Knowledge Substitution: Replace correct domain knowledge with plausible but incorrect info. (7) Reasoning Chain Disruption: Modify or break the logical flow of reasoning. (8) OCR-Specific Errors: Introduce text/number recognition errors. (9) Spatial Position Perturbation: Modify spatial descriptions. (10) Query Intent Redirection: Answer a different but related question. (11) Comparative Relation Inversion: Reverse comparison relationships. (12) Multi-Step Calculation Error: Introduce errors at specific steps in multi-step calculations. (13) Contextual Misattribution: Assign attributes or actions to wrong entities in multi-object scenes. Input: – Question: {question} – Ground Truth Answer: {ground_truth} – Model Answer (Correct): {model_answer} Requirements: 1. Inject 3–5 different hallucination types. Total tags can be more than 5. 2. Use format: <hallucination type="TYPE">hallucinated content</hallucination>. 3. Tags must not be nested or overlapping. Adjacent tags with the SAME type MUST be merged. Adjacent tags with DIFFERENT types are allowed. 4. Make hallucinations plausible and natural. 5. Each tag should cover a natural semantic unit — single word, short phrase (2–3 words), or medium phrase (4–5 words). 6. Maintain text consistency after tag insertion. 7. Diversify hallucination types; do not use a single type throughout. 8. Vary tag lengths; mix single-word and short-phrase tags. 9. Only merge adjacent hallucinated words when they belong to the same type. Output: The complete modified answer with all hallucination tags.
Stage 2b: Hallucination Annotation (for incorrect samples)
You are an expert at identifying and annotating hallucinations in visual question answering responses. Task: The model gave an INCORRECT answer. Identify and annotate the hallucinated parts. Hallucination Types: <i>[Same 12 types as Stage 2a]</i> Dataset Category: {dataset_category} Focus on these types: {primary_types} Input: – Question: {question} – Ground Truth Answer: {ground_truth} – Model Answer (Incorrect): {model_answer} Requirements: 1. Identify parts that differ from the ground truth. 2. Use format: <hallucination type="TYPE">hallucinated content</hallucination>. 3. Tags must not be nested or overlapping. Adjacent tags with the SAME type MUST be merged. Adjacent tags with DIFFERENT types are allowed. 4. Each tag should cover a natural semantic unit — single word, short phrase (2–3 words), or medium phrase (4–5 words). Do not over-split. 5. Maintain text consistency after tag insertion. 6. Diversify hallucination types; do not use a single type throughout. 7. Vary tag lengths; mix single-word and short-phrase tags. 8. Only merge adjacent hallucinated words when they belong to the same type. Output: The model answer with hallucination annotations.

Stage 3: Quality Assessment
You are an expert linguistic quality evaluator. Your task is to evaluate the quality of a generated answer for a Visual Question Answering (VQA) task. Important: The answer contains hallucination tags in the format <hallucination type="TYPE">content</hallucination>. When evaluating quality, you should IGNORE the actual content inside the hallucination tags (as it represents intentionally injected errors). Focus only on evaluating the overall text structure, fluency, and coherence. Evaluate on three dimensions: 1. Clarity (1–5): Is the answer easy to understand? Are the sentences well-structured? 2. Fluency (1–5): Is the language natural and grammatically correct? Does the text flow smoothly? 3. Coherence (1–5): Does the answer structure make sense? Is the text internally consistent in terms of formatting and flow? Input: – Question: {question} – Answer: {answer} Instructions: 1. Analyze the answer based on the three dimensions above. 2. Ignore the factual correctness of content inside hallucination tags — those are intentionally wrong. 3. Evaluate the linguistic quality, structure, and readability of the overall text. 4. Assign a score from 1 to 5 for each dimension. 5. Provide a brief explanation for your scores. 6. Determine if the sample is PASS or FAIL. PASS criteria: all scores ≥ 4. Output: <analysis>...</analysis>, <scores> Clarity/Fluency/Coherence: [1–5] </scores>, <decision> PASS or FAIL with reason </decision>.
Stage 4: Consistency Verification
You are an expert data quality auditor for a Hallucination Dataset. Your task is to verify the accuracy and correctness of hallucination tags injected into an answer. The answer contains tags in the format: <hallucination type="TYPE">content</hallucination>. Allowed Hallucination Types: <i>Perception-based:</i> Object, OCR, Numerical_Attribute, Color_Attribute, Shape_Attribute, Spatial_Attribute. <i>Reasoning-based:</i> Logical_Error, Calculation_Error, Knowledge_Error, Query_Misunderstanding, Numerical_Relation, Spatial_Relation. Verify four aspects: 1. Syntax: Are the tags properly formatted? (Opening and closing tags match, type attribute exists.) 2. Type Correctness: Is the type attribute one of the 12 allowed types listed above? 3. Content Logic: Does the content inside the tag actually represent a hallucination (error) compared to the Original Answer as ground truth? 4. Context: Does the surrounding text flow naturally? Input: – Question: {question} – Original Answer (Ground Truth): {original_answer} – Hallucinated Answer (Target): {hallucinated_answer} Instructions: 1. Compare the Hallucinated Answer against the Original Answer. 2. Check every hallucination tag in the target answer. 3. Verify if the tagged content contradicts the original answer or introduces an error consistent with its type. 4. Verify if the non-tagged parts are consistent with the original answer (unless they were modified to support the hallucination). Output Format: <analysis> Analysis of the tags and their logical correctness compared to the original answer. </analysis> <validation> Syntax_Valid: [Yes/No], Types_Valid: [Yes/No], Logic_Valid: [Yes/No] </validation> <decision> Result: [PASS/FAIL]. Reason: [Brief reason. If FAIL, specify which tag or issue caused it.] </decision>

H.2 Evaluation Prompt

HalluScope-4B/8B use the following three-part prompt for hallucination diagnosis at inference time. The system prompt defines the 12 hallucination types with their descriptions and specifies a strict output format requiring tagged text wrapped in <Tagged_Text> delimiters. The user prompt supplies the image, question, and model response to be analyzed.

System Prompt
You are a precise hallucination detector and classifier for multimodal large language models. Your task is to analyze images and model responses to identify hallucinations. You must tag each hallucination with its specific type. Hallucination Types (use exactly these type names in tags): Perception-based Hallucinations (can be directly verified from the image): – Object: Incorrect identification of objects in visual content. – OCR: Failure in text recognition processes within images. – Numerical_Attribute: Misinterpretation of numerical values in visual elements. – Color_Attribute: Errors in identifying the color. – Shape_Attribute: Misrecognition of object shapes. – Spatial_Attribute: Errors in recognizing the position, orientation, or distance of the object. Reasoning-based Hallucinations (require inference beyond direct perception): – Logical_Error: Errors in reasoning, such as incorrect causal relationships or conflicts in inference steps. – Calculation_Error: Errors in mathematical operations (e.g., addition, subtraction, equation solving). – Knowledge_Error: Applies incorrect domain knowledge or makes unrealistic inferences (e.g., violating common sense or physical laws). – Query_Misunderstanding: Provides incorrect or irrelevant answers due to misunderstanding the query. – Numerical_Relation: Misinterpreting the numerical relationship between objects (e.g., misreading proportions or quantities). – Spatial_Relation: Misunderstanding the spatial, orientation, or distance relationships between objects. IMPORTANT OUTPUT FORMAT REQUIREMENTS: 1. Start with EXACTLY this line: "Here is the response with hallucinated content tagged:" 2. Then use <Tagged_Text> tags to wrap the tagged response 3. Inside <Tagged_Text> tags: – Output the original response with ONLY <hallucination type="TYPE"> tags added – DO NOT modify or change any words in the original response – ONLY add <hallucination type="TYPE"> tags around hallucinated content – Each tag must include the type attribute specifying the hallucination category – If there are no hallucinations, output the original text exactly as is 4. End with </Tagged_Text> 5. DO NOT add any other text, analysis, or explanation 6. ANY OTHER FORMAT WILL BE REJECTED Example Input: prompt given to the model: describe the image model's response: The bright red sports car is parked near a lake. Correct Output Format: Here is the response with hallucinated content tagged: <Tagged_Text> The <hallucination type="Color_Attribute">bright red</hallucination> <hallucination type="Object">sports</hallucination> car is <hallucination type="Spatial_Attribute">parked near a lake</hallucination>. </Tagged_Text> INCORRECT Outputs (DO NOT DO THESE): – Any text before "Here is the response with hallucinated content tagged:" – Any text between the header and <Tagged_Text> – Any text after </Tagged_Text> – Any explanatory text or analysis – Any modification to the original text – Any additional formatting or tags besides <hallucination type="TYPE"> Remember: – Tag each hallucination separately and specifically with its type – Keep the content of the original text strictly unchanged – Use exactly the type names listed above (with underscores) – Perception-based types (Object, OCR, Numerical_Attribute, Color_Attribute, Shape_Attribute, Spatial_Attribute): Use when the error can be directly verified from the image – Reasoning-based types (Numerical_Relation, Spatial_Relation, Logical_Error, Calculation_Error, Knowledge_Error, Query_Misunderstanding): Use when the error involves comparison, reasoning, or inference beyond direct observation
User Prompt
<image> Here is the prompt given to the model: {question} Here is the model's response: {answer} Please analyze the image and add <hallucination> tags to any hallucinated content in the model's response. Remember to tag each hallucinated content separately!

type classification; CRITIC-ClS further incorporates type annotations into both span marking and correction instructions; and HalluScope uses HalluScope-8B to generate interpretable explanations that inform the target model what is wrong, why it is wrong, and how to correct it.

System Prompt
You are a careful visual question answering assistant. You will be given your previous answer with hallucination annotations marked. Please re-examine the image, consider the marked hallucinations, and provide a corrected answer.
Self-Refine
User: Your previous answer to this question was incorrect. Please try again. Please: 1. Carefully re-examine the image 2. Think step by step about what you actually see 3. Provide a corrected answer Question: {question} Now please provide your corrected answer based on what you actually observe in the image.
CRITIC-Det
User: Your previous answer to this question had some issues. A reviewer has identified the following problems: <i>[Generated by generate_structured_summary_from_detection():]</i> Question: {question}. Please re-examine the image carefully, correct the issues listed above, and provide a new answer.
CRITIC-ClS
User: Your previous answer to this question had some issues. A reviewer has identified the following problems: <i>[Generated by generate_structured_summary():]</i> Question: {question}. Please re-examine the image carefully, correct the issues listed above, and provide a new answer.
HalluScope
User: Your previous answer to this question had some issues. Here is feedback from a reviewer: {suggestion} <i>[The suggestion is generated by HalluScope-8B using the <Suggestion> variant of the evaluation prompt: 1–3 sentences explaining what is wrong, why it is wrong based on the image, and how to correct it.]</i> Question: {question}. Please re-examine the image carefully based on the feedback above and provide a corrected answer.

H.3 Feedback Prompt Templates

All four feedback modes share the same system prompt and differ only in the user message. The diagnostic granularity increases progressively: Self-Refine simply informs the target model that its previous answer was incorrect without any external diagnosis; CRITIC-Det provides detection-level feedback by marking hallucinated spans with structured correction instructions but without